

CSE 466 Fundamentals of Blockchain

Spring 2025

Assoc. Prof. Hisham Dahshan

Course Information Policies

- **Grading:**

- Weeks 3:7th Midterm: **30%** (15% Midterm + 15% Labs & Quiz)
- Weeks 8:12th Midterm: **20%** (10% Midterm + 10% Labs & Quiz)
- Pre-Final: Project: **10%** (Topics by week #3 Submission & Discussion during last week of semester)
- Final Exam: **40%**

- **Attendance:**

- Attendance for lectures and labs is **mandatory**

Course Contents:

Week #	Description
1	Intro to cryptography & cryptocurrencies
2	Bitcoin
3	Classical Consensus Protocols
4	Consensus in the internet setting
5	Accountability and incentives in consensus: proof-of-stake
6	Ethereum
8	Stablecoins and lending protocols
9	Decentralized exchanges
10	Maximal Extractable Value (MEV)
11	Privacy: de-anonymizing the blockchain and mixing
13	zk-SNARKs: what they are and how they are used
14	Constructing a preprocessing zk-SNARK
15	Scaling the blockchain: payment channels and state channels
16	Final Exam

What is a blockchain for?

a blockchain provides
coordination between many parties,
when there is no single trusted party

if trusted party exists $\Rightarrow$ no need for a blockchain

[financial systems: often no trusted party]

Blockchains: what is the new idea?

2009

Bitcoin

Several innovations:

- A practical **public append-only data structure**, secured by replication and incentives
- A **fixed supply asset (BTC)**. Digital payments, and more.

An **append-only data structure** is a type of data storage where new data can only be added to the end of the existing structure. Once data is written, it cannot be modified or deleted.

A **Fixed Supply Asset** in cryptocurrency refers to a digital asset that has a set maximum supply cap. Once this cap is reached, no more units of the asset can be created. **21 million coins in BTC**

Blockchains: what is the new idea?

2009

2015

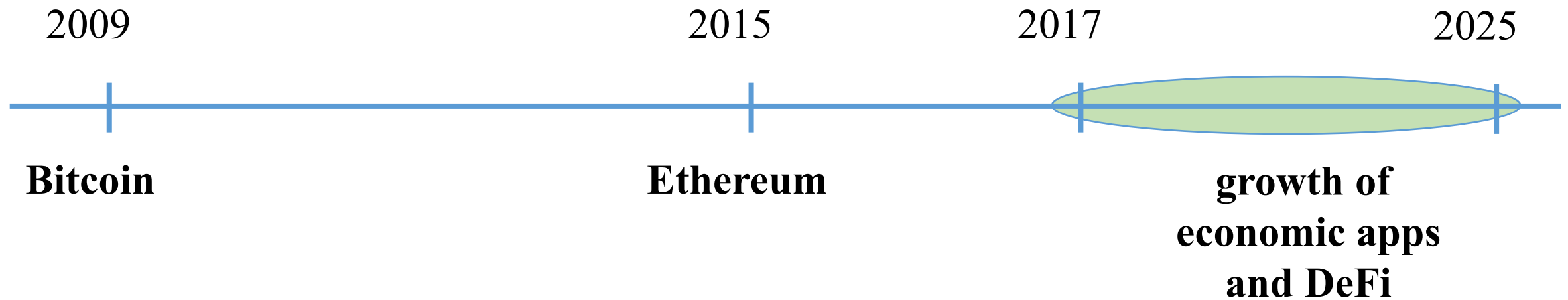
Bitcoin

Ethereum

Several innovations:

- **Blockchain computer:** a fully programmable environment
⇒ public programs that manage digital and financial assets
- **Composability:** applications running on chain can call each other

Blockchains: what is the new idea?



So what is this good for?

- (1) Basic application: a digital currency (stored value)
- Current largest: Bitcoin (2009), Ethereum (2015), Solana (2020)
 - Global: accessible to anyone with an Internet connection

What else is it good for?

(2) Decentralized applications (DAPPs)

- **DeFi:** financial instruments managed by public verifiable programs
 - examples: stablecoins, lending, exchanges,
- **Decentralized organizations (DAOs):** (decentralized governance)
 - DAOs for investment, for donations, for collecting art, etc.
- **x402:** AI payments -- agents and crawlers paying for services. x402 is an open payment standard that lets AI agents and APIs pay per use—no accounts, subscriptions, or API keys required.

(3) New programming model: writing decentralized programs

Assets managed by DAPPs

Aave v3	\$41.8B	lending	multiple chains
LIDO	\$39.5B	staking	Ethereum
WBTC	\$14.7B	bridging	Ethereum
Morpho (663 lines of Solidity code)	\$7.1B	lending	multiple chains
Uniswap v3	\$3B	exchange	multiple chains

Transaction volumes

24h volume

Sep. 2025

 Bitcoin • BTC	\$13.5B
 Ethereum • ETH	\$2.35B
 Solana	\$8.95B

Cost to send USD internationally:

\$44 via wire transfer

<\$0.01 via USDC on Base

What is a blockchain?

Data Availability / Consensus Layer:

This is the **foundation** of the blockchain. It provides two crucial services:

- 1.Consensus** — ensures everyone agrees on the same chain of blocks (e.g., Proof-of-Work, Proof-of-Stake).
- 2.Data Availability** — makes sure that all the transaction data is visible and available to all participants.

Data Availability / Consensus Layer

What is a blockchain?

Sequencer: Orders Transactions

- Before execution, transactions need to be ordered.
- The **sequencer** decides the order in which transactions are processed.
- This ensures consistency — otherwise, two different users might see different histories.

Sequencer: orders transactions

Data Availability / Consensus Layer

What is a blockchain?

Execution Engine (blockchain computer)

- Sometimes called the **Virtual Machine** (e.g., Ethereum Virtual Machine).
- It **executes the code** of smart contracts.
- This is the computational layer of the blockchain.

Execution engine (blockchain computer)

Sequencer: orders transactions

Data Availability / Consensus Layer

What is a blockchain?

Applications (DApps, smart contracts)

- This is where **Decentralized Applications (DApps)** and **smart contracts** live.
- These are the actual programs that run on the blockchain (e.g., decentralized finance apps, NFT marketplaces, supply chain apps).

applications (DAPPs, smart contracts)

Execution engine (blockchain computer)

Sequencer: orders transactions

Data Availability / Consensus Layer

What is a blockchain?

User Facing Tools (cloud servers): This is what end-users interact with.

- Examples: wallets, dashboards, exchanges, or cloud-hosted services that connect people to blockchain applications.

user facing tools (cloud servers)

applications (DAPPs, smart contracts)

Execution engine (blockchain computer)

Sequencer: orders transactions

Data Availability / Consensus Layer

Consensus layer (informal)

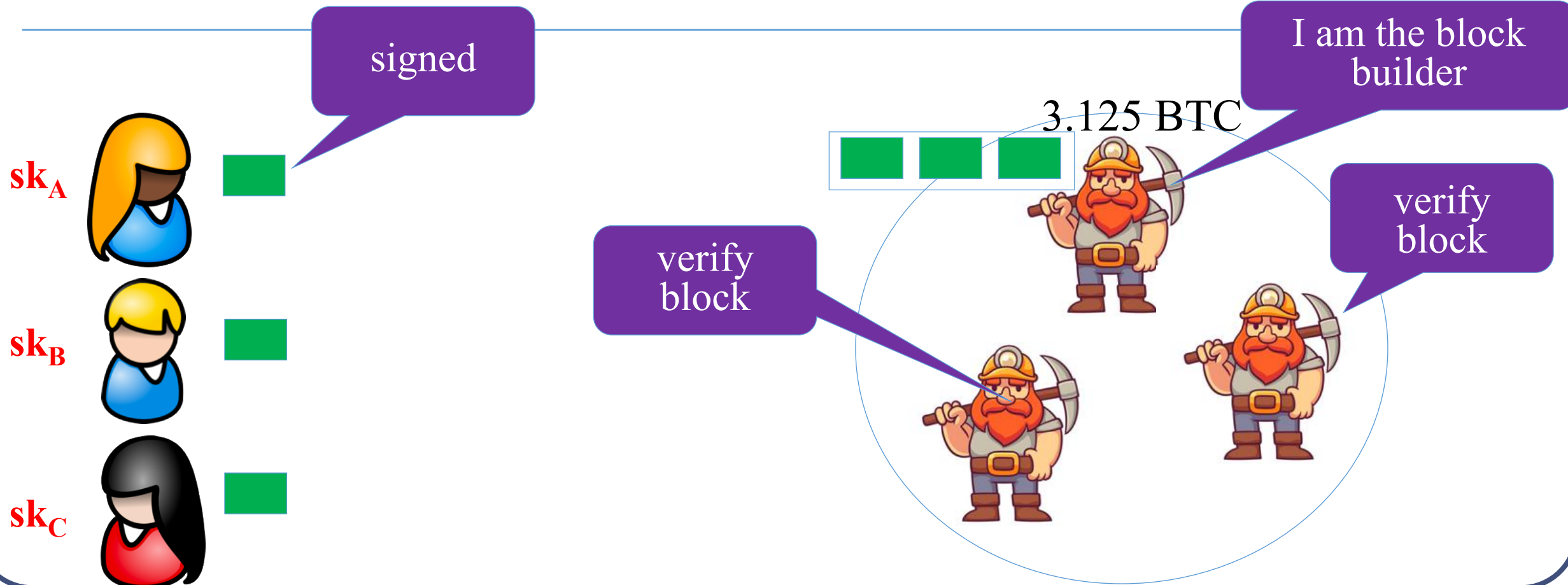
A public append-only data structure:

achieved by replication

- **Persistence:** once added, data can never be removed*
- **Safety:** all honest participants have the same data**
- **Liveness:** honest participants can add new transactions
- **Open(?):** anyone can add data (no authentication)

How are blocks added to chain?

blockchain

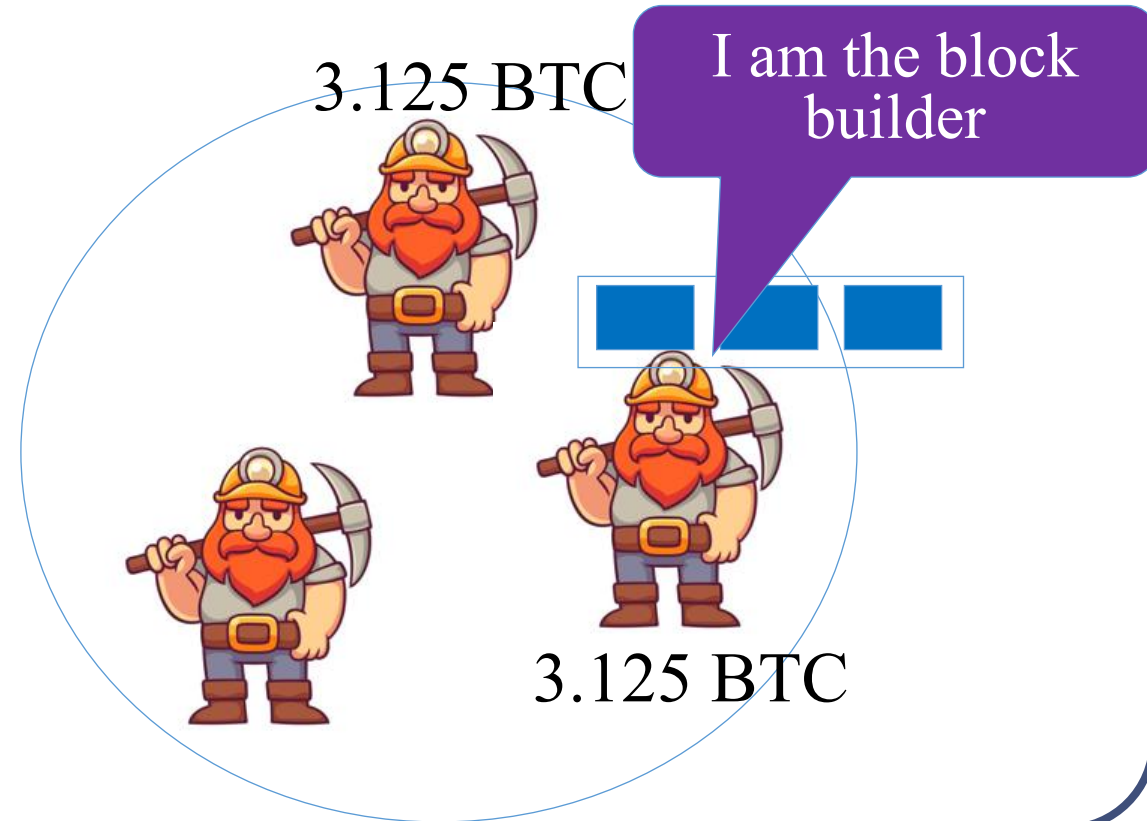
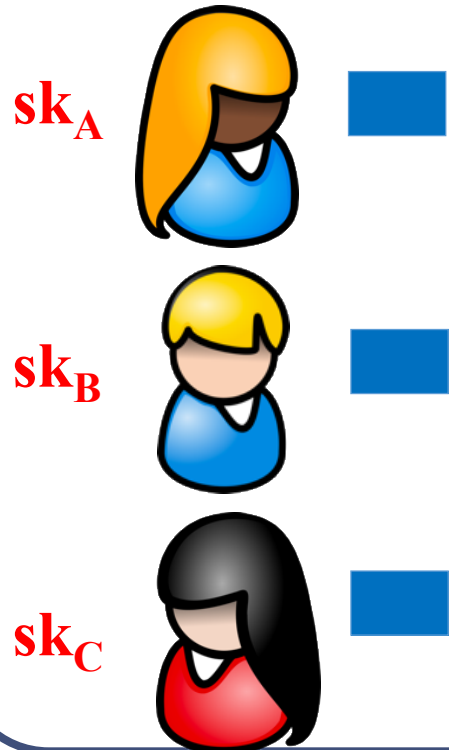


How are blocks added to chain?

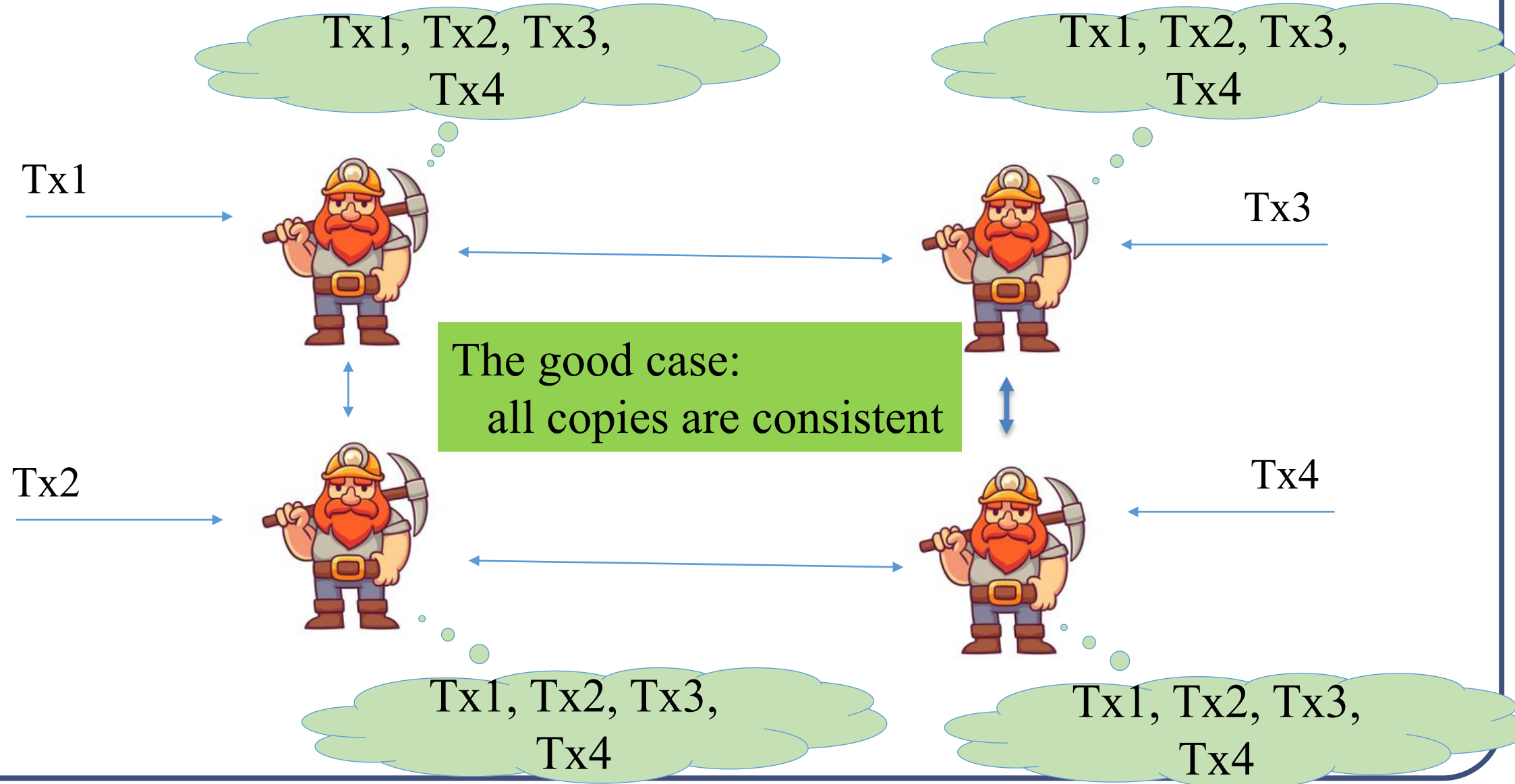
blockchain



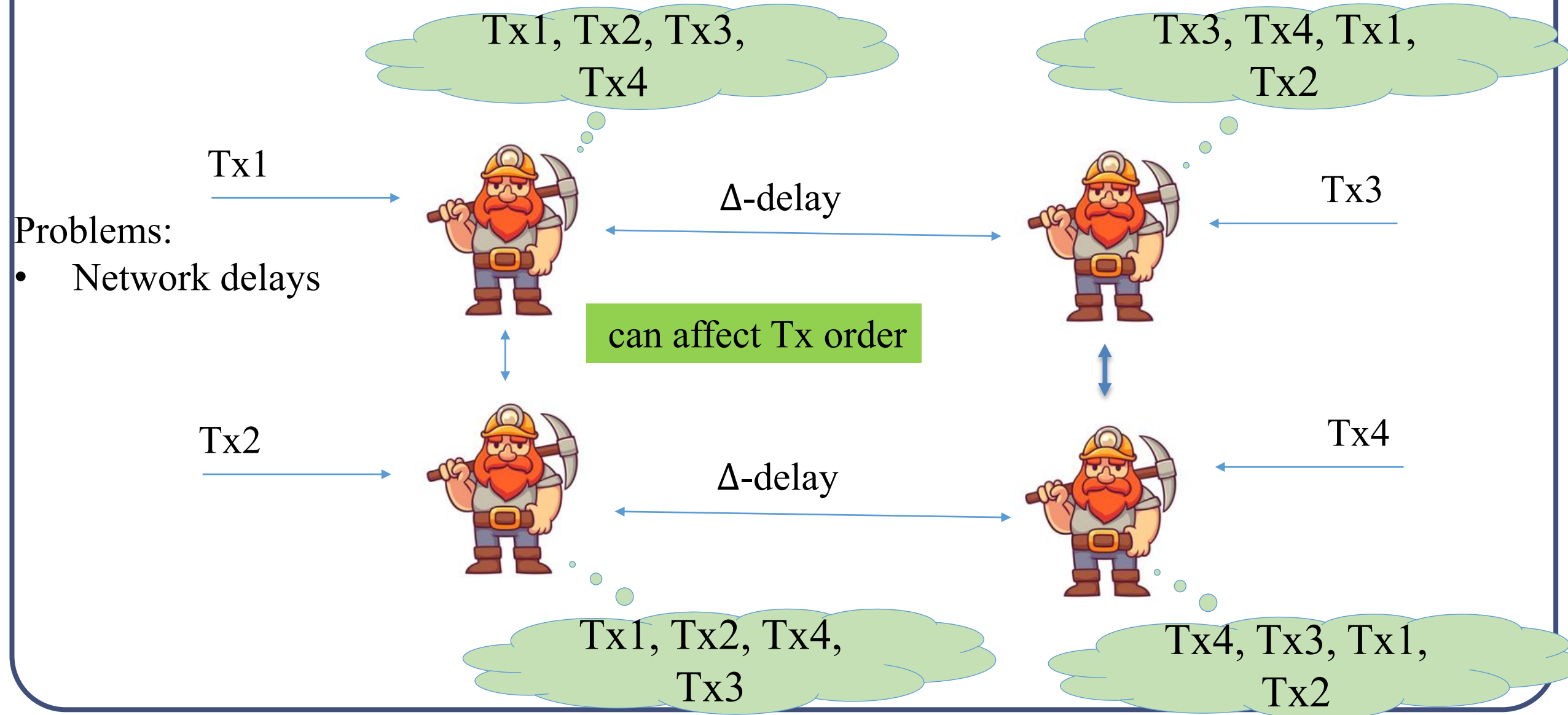
...



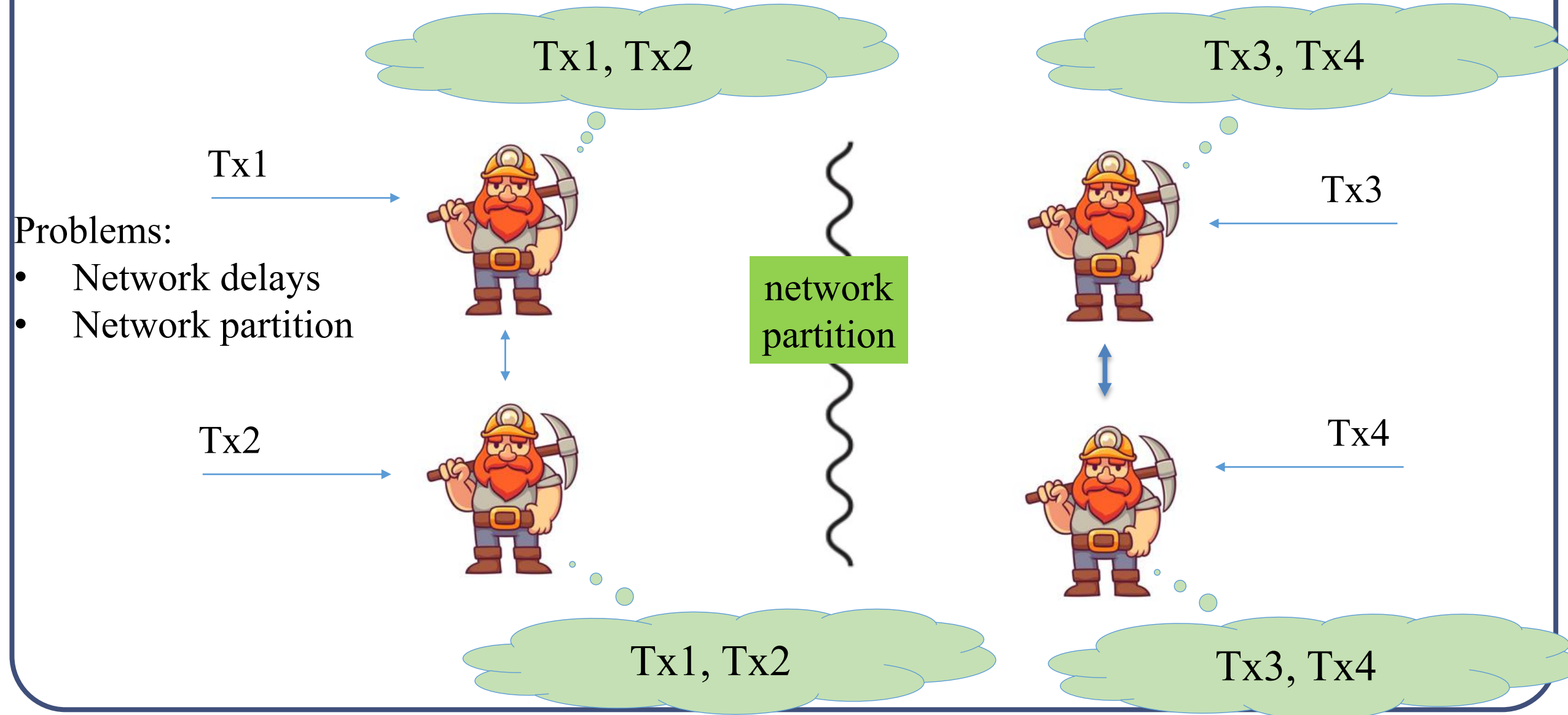
Why is consensus a hard problem?



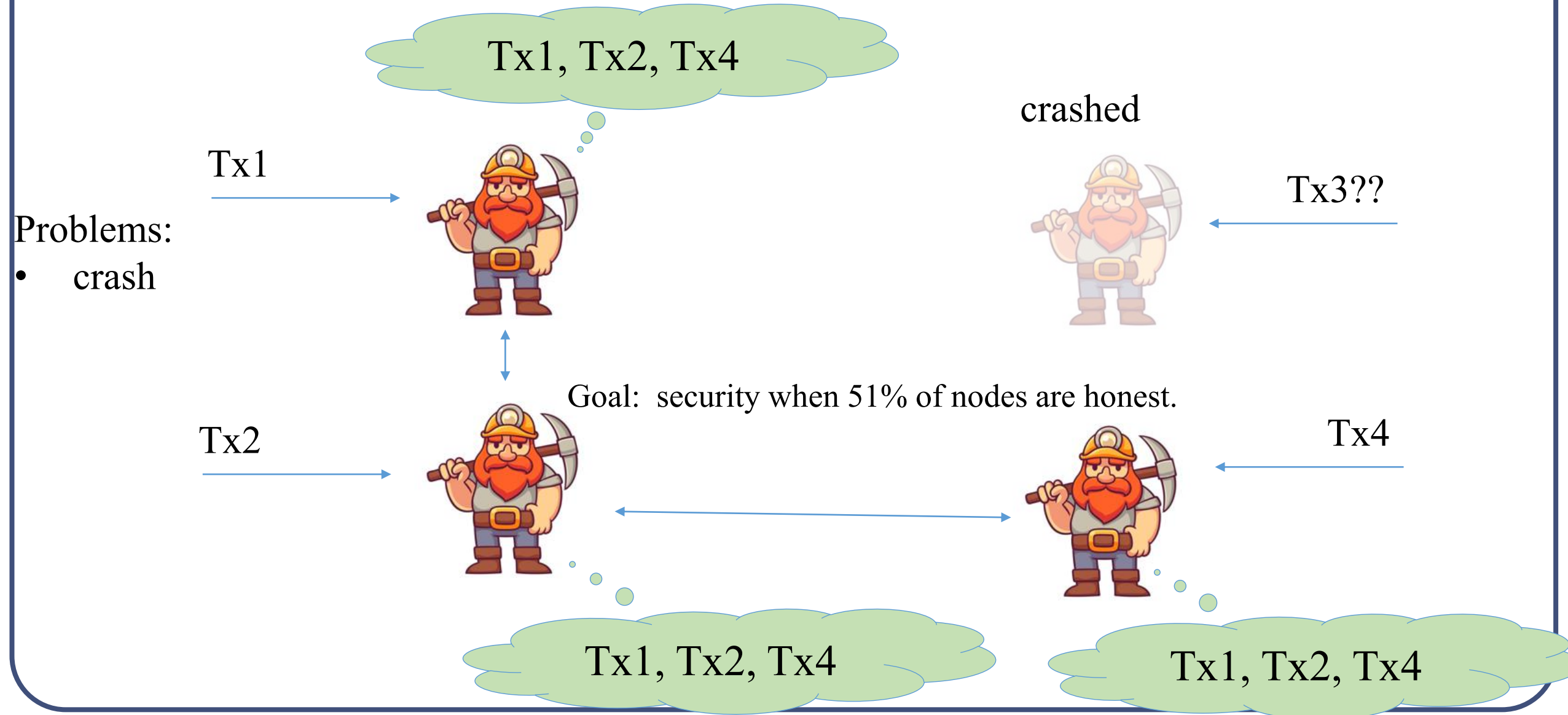
Why is consensus a hard problem?



Why is consensus a hard problem?



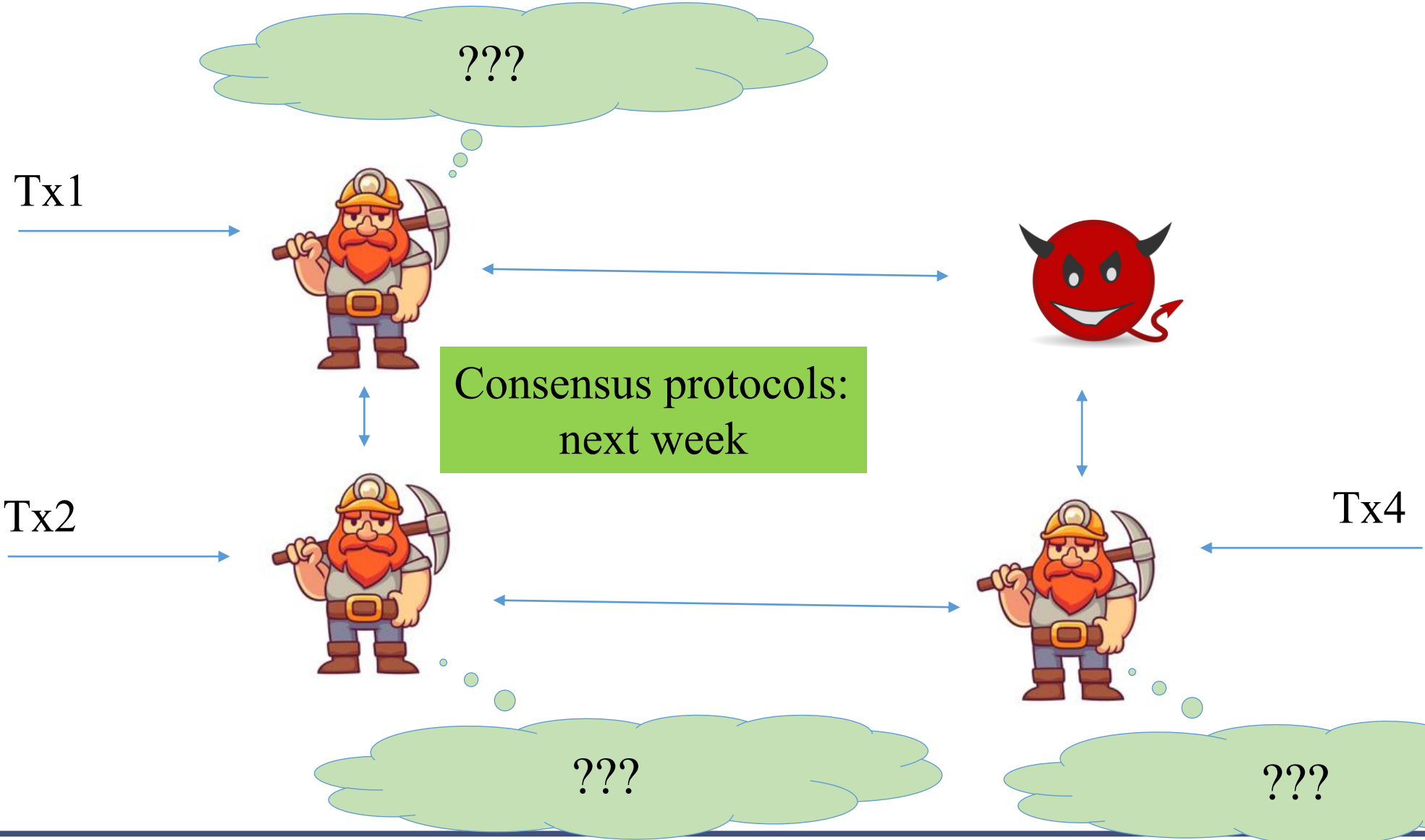
Why is consensus a hard problem?



Why is consensus a hard problem?

Problems:

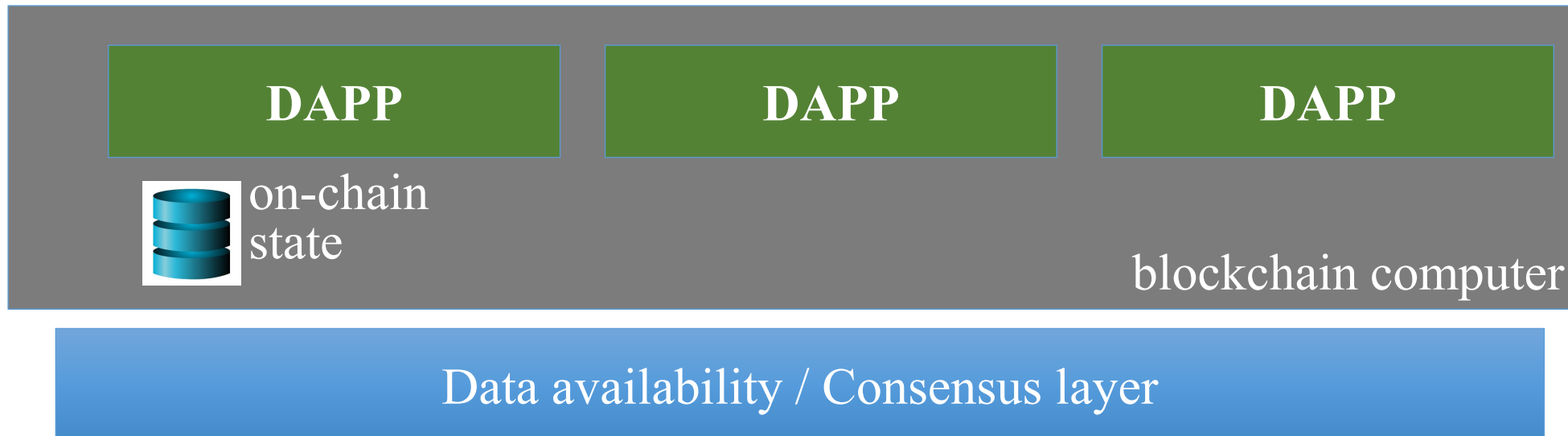
- crash
- malice



Next layer: the blockchain computer

Decentralized applications (DAPPs):

- Run on blockchain: code and state are written on chain
- Accept Tx from users $\Rightarrow$ state transitions are recorded on chain



Next layer: the blockchain computer

Top layer: user facing servers

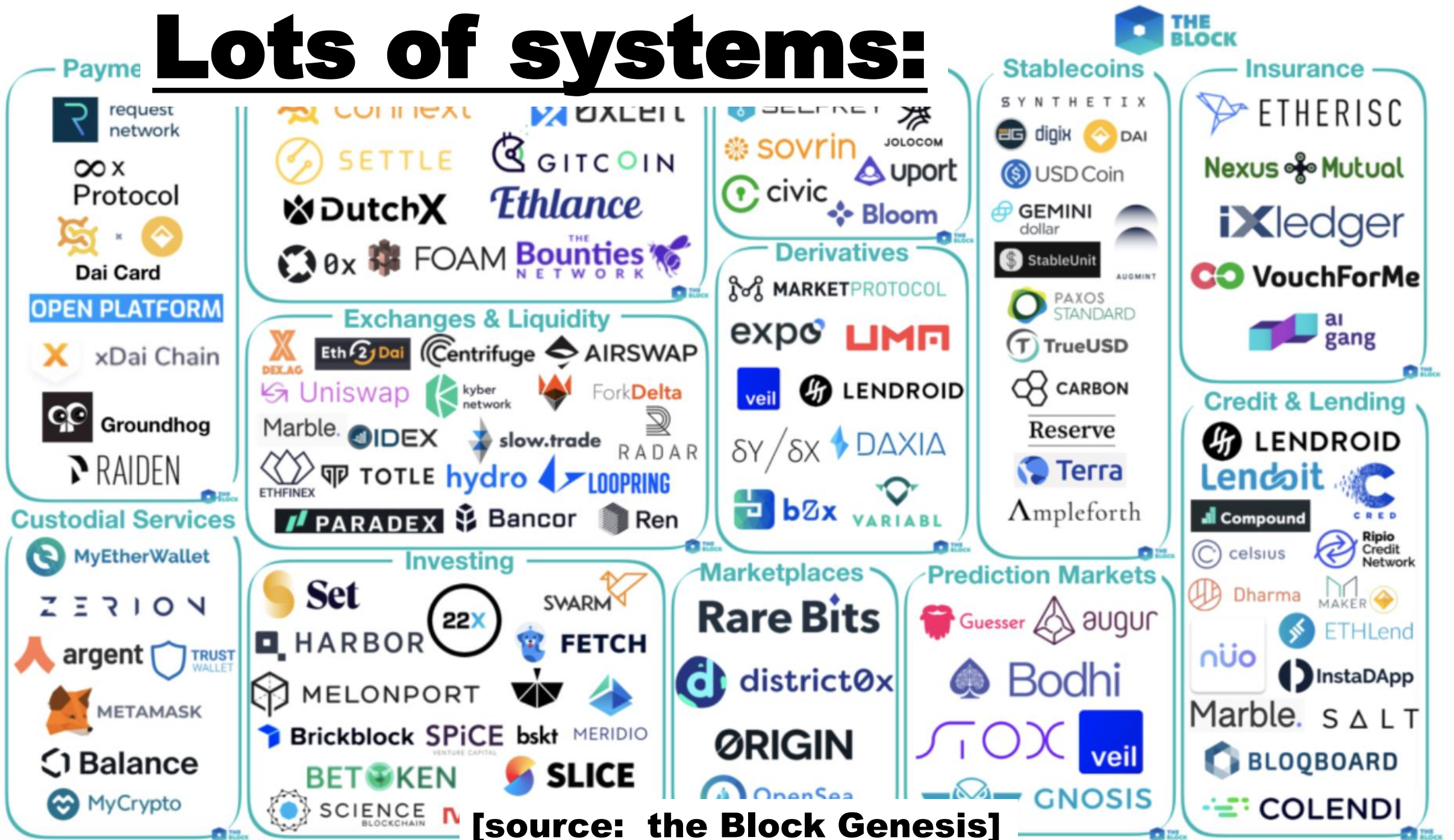


end user



Data availability / Consensus layer

Lots of systems:



[source: the Block Genesis]

Let's get started ...

Cryptography Background

(1) cryptographic hash functions

An efficiently computable function $H: M \rightarrow T$

where $|M| \gg |T|$



Collision resistance

Def: a collision for $H: M \rightarrow T$ is pair $x \neq y \in M$ s.t. $H(x) = H(y)$

$|M| \gg |T|$ implies that many collisions exist

Def: a function $H: M \rightarrow T$ is collision resistant if it is “hard” to find even a single collision for H (we say H is a CRF)

Example: **SHA256:** $\{x : \text{len}(x) < 2^{64} \text{ bytes}\} \rightarrow \{0,1\}^{256}$

(output is 32 bytes)

Application: committing to data on a blockchain



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

Alice has a large file m . She posts $h = H(m)$ on the blockchain (32 bytes)

Bob reads h . Later he learns m' s.t. $H(m') = h$

H is a CRF $\Rightarrow$ Bob is convinced that $m' = m$
(otherwise, m and m' are a collision for H)

We say that $h = H(m)$ is a **binding commitment** to m

(note: h may leak information about m)

Committing to a list (of transactions)

Alice has $S = (m_1, m_2, \dots, m_n)$

32 bytes

Goal:

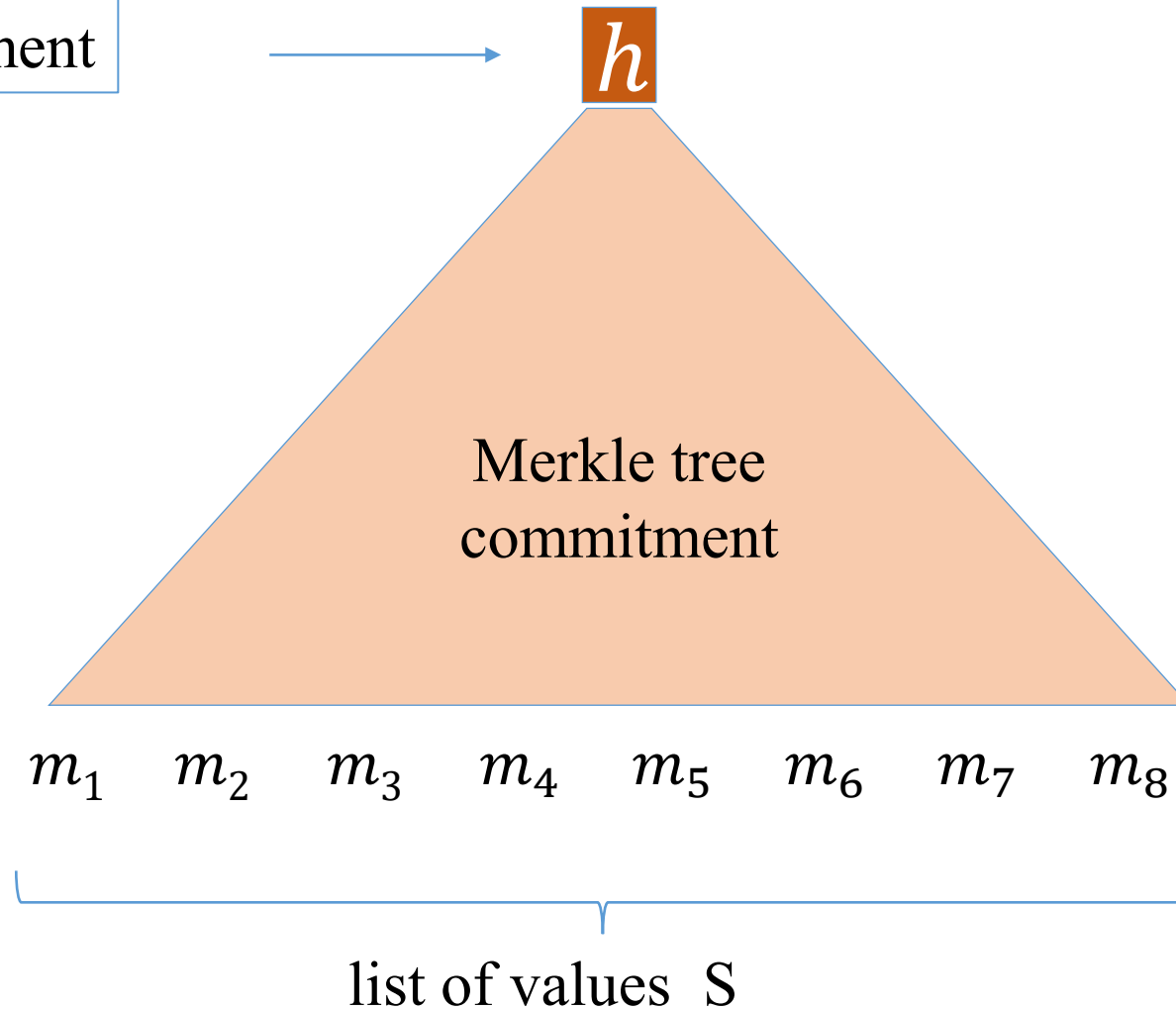
- Alice posts a short binding commitment to S , $h = \text{commit}(S)$
- Bob reads h . Given $(m_i, \text{proof } \pi_i)$ can check that $S[i] = m_i$
Bob runs $\text{verify}(h, i, m_i, \pi_i) \rightarrow \text{accept/reject}$

security: adv. cannot find (S, i, m, π) s.t. $m \neq S[i]$ and

$\text{verify}(h, i, m, \pi) = \text{accept}$ where $h = \text{commit}(S)$

Merkle tree (Merkle 1989)

commitment



Goal:

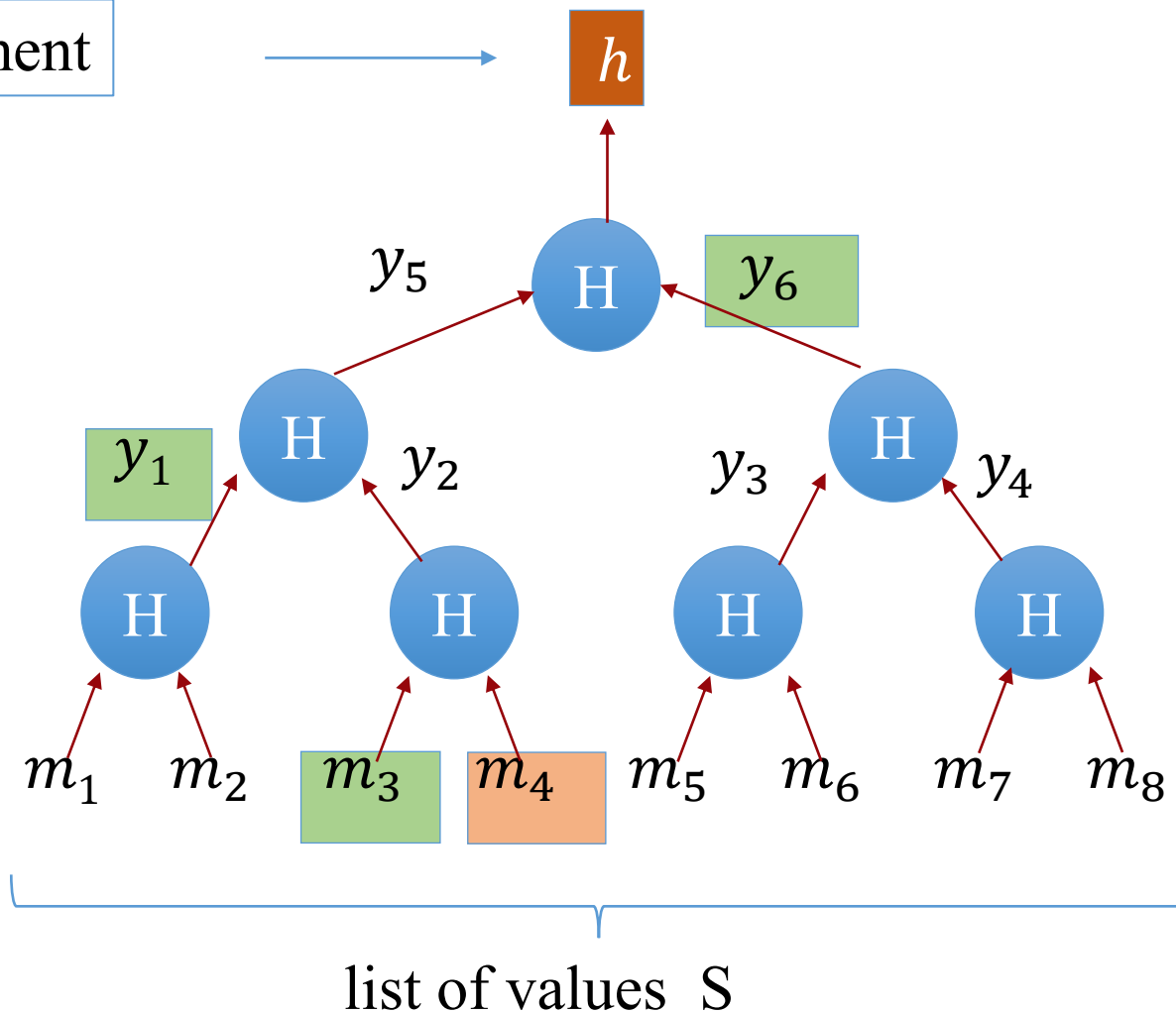
- commit to list S of size n
- Later prove $S[i] = m_i$

Merkle tree

(Merkle 1989)

[simplified]

commitment



Goal:

- commit to list S of size n
- Later prove $S[i] = m_i$

To prove $S[4] = m_4$,

proof $\pi = (m_3, y_1, y_6)$

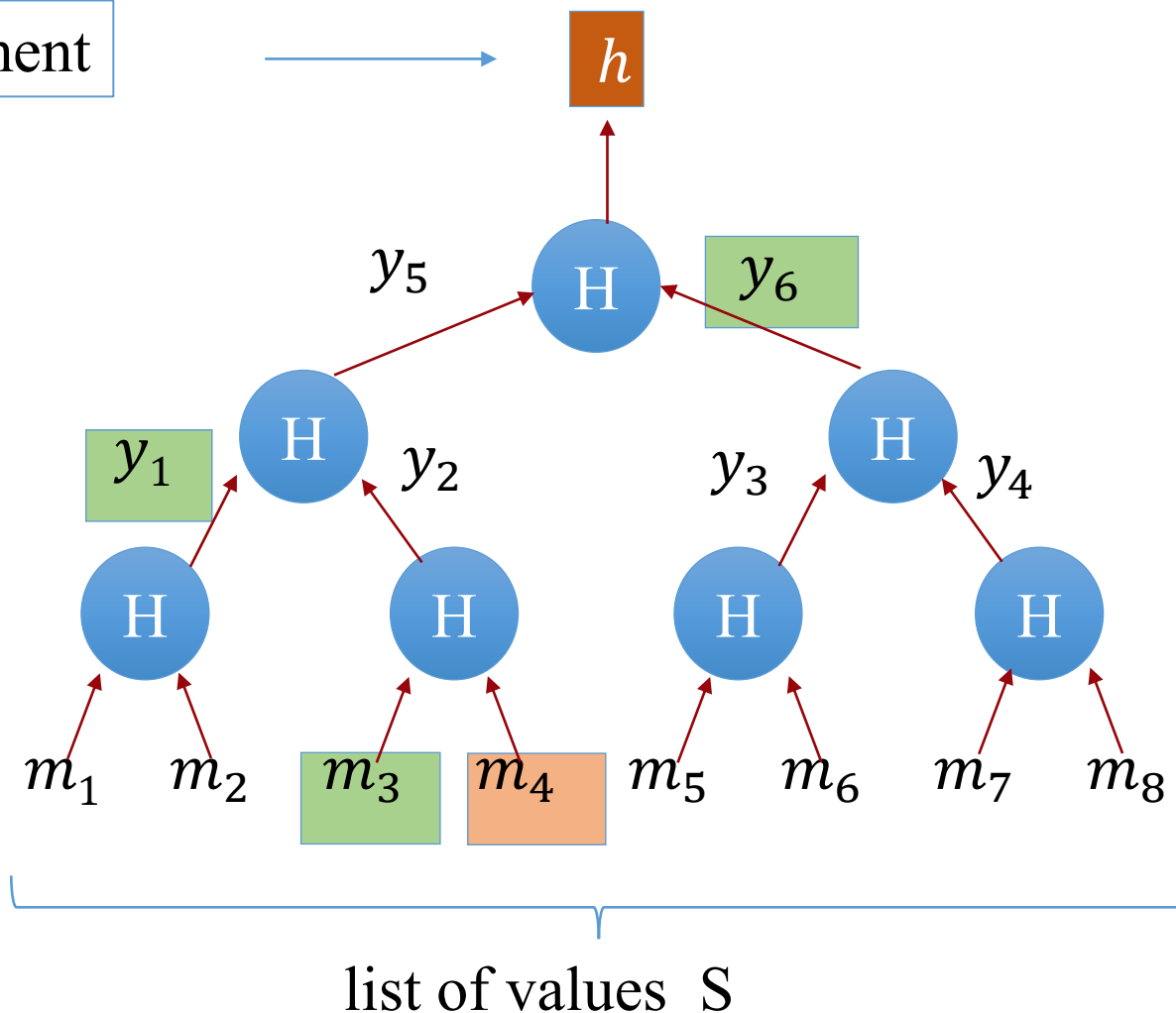
length of proof: $\log_2 n$

Merkle tree

(Merkle 1989)

[simplified]

commitment



To prove $S[4] = m_4$,

proof $\pi = (m_3, y_1, y_6)$

Bob does:

$$y_2 \leftarrow H(m_3, m_4)$$

$$y_5 \leftarrow H(y_1, y_2)$$

$$h' \leftarrow H(y_5, y_6)$$

accept if $h = h'$

Merkle tree (Merkle 1989)

Thm: For a given n : if H is a CRF then
adv. cannot find (S, i, m, π) s.t. $|S| = n$, $m \neq S[i]$,
 $h = \text{commit}(S)$, and $\text{verify}(h, i, m, \pi) = \text{accept}$

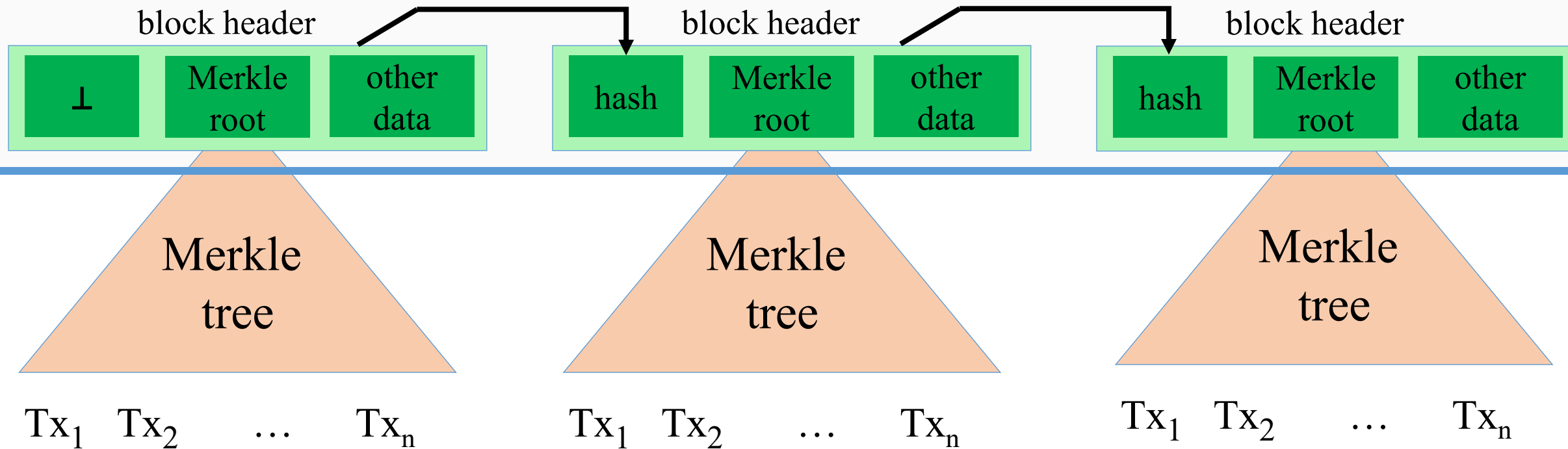
(to prove, prove the contra-positive)

How is this useful? To post a block of transactions S on chain
suffices to only write $\text{commit}(S)$ to chain. Keeps chain small.

$\Rightarrow$ Later, can prove contents of every Tx.

Abstract block chain

blockchain



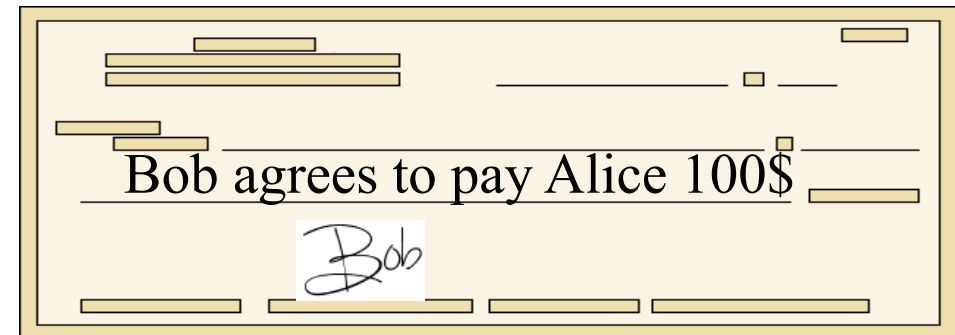
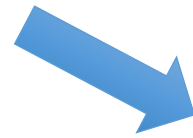
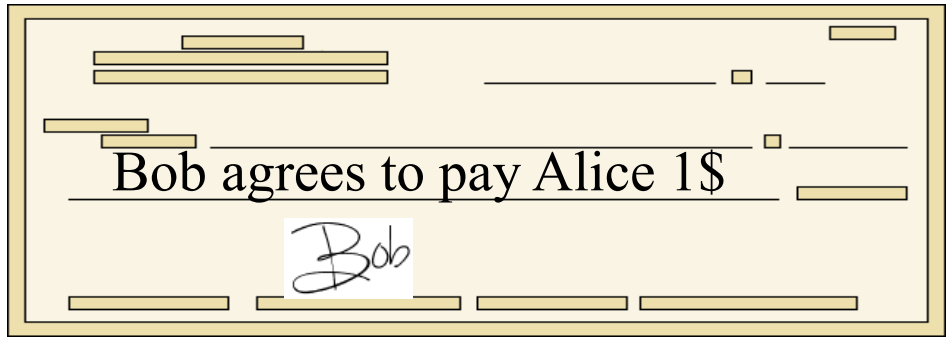
Merkle proofs are used to prove that a Tx is “on the block chain”

Cryptography background: Digital Signatures

How to authorize a transaction

Signatures

Physical signatures: bind transaction to author



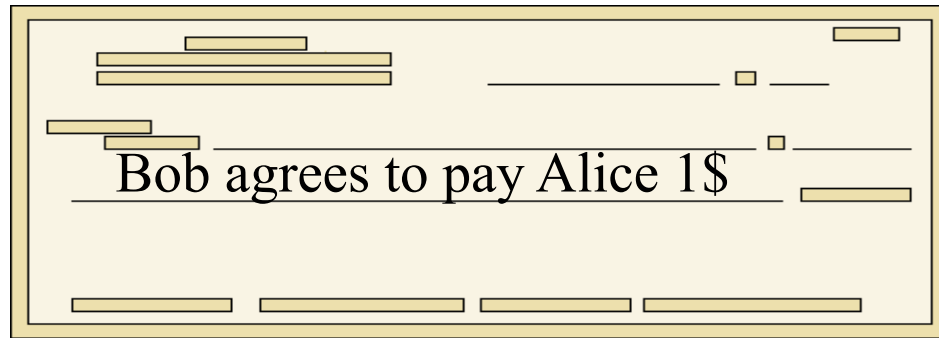
Problem in the digital world:

anyone can copy Bob's signature from one doc to another

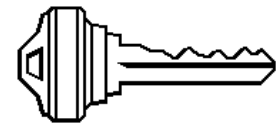
Digital signatures

Solution: make signature depend on document

Signer



signature



secret signing
key (sk)

signing
algorithm

Verifier



'accept'
or
'reject'



public verification
key (pk)

Digital signatures: syntax

Def: a signature scheme is a triple of algorithms:

- **Gen()**: outputs a key pair (pk, sk)
- **Sign**(sk, msg) outputs sig. σ
- **Verify**(pk, msg, σ) outputs 'accept' or 'reject'

Secure signatures: (informal)

Adversary who sees signatures **on many messages** of his choice, cannot forge a signature on a new message.

Families of signature schemes

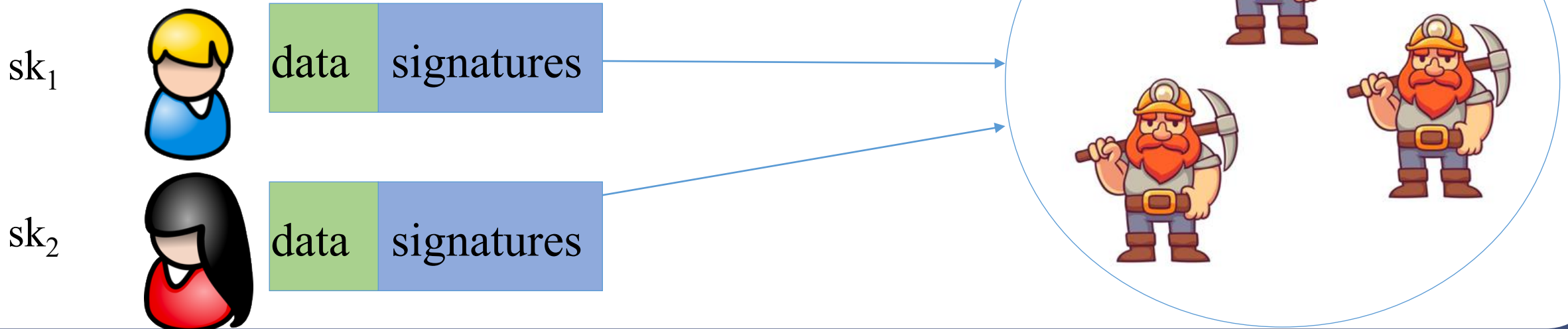
1. RSA signatures (old ... not used in blockchains):
 - long sigs and public keys (≥ 256 bytes), fast to verify
2. Discrete-log signatures: Schnorr and ECDSA (Bitcoin, Ethereum)
 - short sigs (48 or 64 bytes) and public key (32 bytes)
3. BLS signatures: 48 bytes, aggregatable, easy threshold
Boneh–Lynn–Shacham (Ethereum, Chia, Dfinity)
4. Post-quantum signatures (ML-DSA): long (≥ 600 bytes)

Module-Lattice-Based

Signatures on the blockchain

Signatures are used everywhere:

- ensure Tx authorization,
- governance votes,
- consensus protocol votes.



Questions?